

SENA

Servicio Nacional de Aprendizaje

TALLER N° 2

HERENCIA

Programación Orientada a Objetos en Java

Pilar de la POO: Herencia

Lenguaje: Java

Nivel: Básico

Ejercicios: 5 ejercicios prácticos + 1 reto integrador

Metodología: Constructivista (aprender-haciendo)

Modalidad: Clases y métodos main en archivos separados

ELABORADO POR

Carlos Arturo Barrientos Suares

*Ingeniero de Sistemas · Especialista en Desarrollo de Software · Máster en Ingeniería de Software y Sistemas
Informáticos*

Marco conceptual del taller

¿Qué es la herencia?

Piensa en tu familia. Tú **heredaste** características de tus padres: quizás el color de ojos, el tono de la piel, algunos rasgos de personalidad. Pero además tienes cosas que solo tú tienes. En programación es exactamente igual: una clase **hija** hereda atributos y métodos de una clase **padre**, y además puede tener los suyos propios. Modela relaciones del tipo "**es un**": un Perro *es un* Animal; un Carro *es un* Vehículo.

¿Cómo se aplica en Java?

En Java se usa la palabra reservada **extends** para indicar que una clase hereda de otra. La clase hija recibe todos los atributos y métodos del padre, y puede:

- Agregar nuevos atributos o métodos propios.
- **Sobrescribir** métodos del padre usando la anotación `@Override`.
- Invocar al constructor del padre con **super()**.

Los atributos del padre deben ser **protected** (no **private**) si queremos que las hijas puedan acceder a ellos directamente.

¿Por qué es tan importante?

1. **Reutilización de código:** el código del padre no se reescribe en cada hija.
2. **Organización jerárquica:** las clases se organizan de lo general a lo particular, reflejando el mundo real.
3. **Extensibilidad:** agregar una nueva variante (nueva hija) no requiere modificar las existentes.
4. **Mantenimiento centralizado:** un cambio en la lógica común se hace una sola vez.
5. **Base para el polimorfismo:** sin herencia no existiría el polimorfismo que verás en el Taller 4.

REGLA DE ORO

Usa herencia **solo cuando exista una relación natural "es un"**. Si no puedes decir "una X es un Y" con sentido, entonces no debes heredar. Por ejemplo: un Perro *es un* Animal (correcto), pero una Rueda **NO es un** Carro (incorrecto — la rueda es *parte de* un carro, no una especialización).

Ejercicio 1: Mascotas de una Veterinaria

SITUACIÓN PROBLEMA

La veterinaria "Amigos Peludos" de Cúcuta te pide un programa para registrar las mascotas que atiende. Todos los animales que llegan tienen algo en común: tienen **nombre**, **edad**, pueden **comer** y **dormir**. Sin embargo, cada especie **hace un sonido distinto**: el perro ladra, el gato maúlla, el pájaro canta.

Tu tarea: diseñar una clase padre **Animal** y las clases hijas **Perro** y **Gato** aprovechando la herencia.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Antes de escribir código, reflexiona:

1. Si en el futuro llegan Conejos, Hamsters y Loros a la veterinaria, ¿deberías reescribir el método `comer()` en cada clase?
2. ¿Qué ventaja tiene poner `comer()` y `dormir()` en la clase padre?
3. ¿Por qué el sonido debe ser diferente en cada hija y no en el padre?

Solución guiada

Nota que los atributos del padre son **protected** (no **private**) para que las clases hijas puedan usarlos directamente. Además, cada hija usa **super()** para invocar el constructor del padre.

Archivo: `Animal.java`

```
public class Animal {
    protected String nombre;
    protected int edad;

    public Animal(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public void comer() {
        System.out.println(nombre + " esta comiendo.");
    }

    public void dormir() {
        System.out.println(nombre + " esta durmiendo.");
    }

    public void hacerSonido() {
        System.out.println(nombre + " emite un sonido.");
    }
}
```

Archivo: Perro.java

```
public class Perro extends Animal {

    public Perro(String nombre, int edad) {
        super(nombre, edad); // Llama al constructor del padre
    }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " dice: Guau guau!");
    }

    public void perseguirCola() {
        System.out.println(nombre + " persigue su cola.");
    }
}
```

Archivo: Gato.java

```
public class Gato extends Animal {

    public Gato(String nombre, int edad) {
        super(nombre, edad);
    }

    @Override
    public void hacerSonido() {
        System.out.println(nombre + " dice: Miau!");
    }

    public void ronronear() {
        System.out.println(nombre + " ronronea felizmente.");
    }
}
```

Archivo: MainVeterinaria.java

```
public class MainVeterinaria {
    public static void main(String[] args) {
        Perro firulais = new Perro("Firulais", 4);
        Gato michi = new Gato("Michi", 2);

        firulais.comer(); // Heredado del padre
        firulais.hacerSonido(); // Sobrescrito
        firulais.perseguirCola(); // Propio de Perro

        System.out.println("---");

        michi.dormir(); // Heredado del padre
        michi.hacerSonido(); // Sobrescrito
        michi.ronronear(); // Propio de Gato
    }
}
```

BENEFICIO APLICADO

Los métodos *comer()* y *dormir()* se escribieron **una sola vez** en la clase padre. Si mañana se agregan más animales (Conejo, Caballo, Loro), todos heredarán esos métodos **sin duplicar código**. Este principio se llama **DRY: Don't Repeat Yourself** (no te repitas).

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Crea una tercera clase hija **Pajaro** que también extienda de *Animal*. Debe sobrescribir *hacerSonido()* con el mensaje "Pio pio!" y agregar un método propio **volar()**. Añade un pájaro al main y prueba sus métodos. **Reto extra:** agrega en la clase padre *Animal* un atributo **especie** y muéstralo en un método *presentarse()*.

Ejercicio 2: Nómina de una Empresa de Cúcuta

SITUACIÓN PROBLEMA

Una empresa en Cúcuta tiene distintos tipos de empleados. Todos tienen **nombre**, **documento** y **salario base**. Pero cada uno recibe pagos distintos:

- Un **Vendedor** gana su salario base **más comisiones** por ventas.
- Un **Gerente** gana su salario base **más una bonificación fija**.

Tu tarea: diseñar la clase padre **Empleado** y las hijas **Vendedor** y **Gerente**, cada una con su propio cálculo de salario.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Reflexiona:

1. Si el método *calcularSalario()* solo estuviera en las hijas, ¿podríamos poner el *mostrarInfo()* en el padre?
2. ¿Qué ventaja tiene que el método *mostrarInfo()* viva solo en el padre, aunque cada empleado calcule diferente su salario?
3. Si mañana se agrega el rol de **Aprendiz** (que gana solo el salario base), ¿qué tan poco código tendrías que escribir?

Solución guiada

Archivo: **Empleado.java**

```
public class Empleado {
    protected String nombre;
    protected String documento;
    protected double salarioBase;

    public Empleado(String nombre, String documento, double salarioBase) {
        this.nombre = nombre;
        this.documento = documento;
        this.salarioBase = salarioBase;
    }

    public double calcularSalario() {
        return salarioBase;
    }

    public void mostrarInfo() {
        System.out.println("Empleado: " + nombre);
        System.out.println("Documento: " + documento);
        System.out.println("Salario a pagar: $" + calcularSalario());
    }
}
```

Archivo: Vendedor.java

```
public class Vendedor extends Empleado {
    private double comision;

    public Vendedor(String nombre, String documento,
                    double salarioBase, double comision) {
        super(nombre, documento, salarioBase);
        this.comision = comision;
    }

    @Override
    public double calcularSalario() {
        return salarioBase + comision;
    }
}
```

Archivo: Gerente.java

```
public class Gerente extends Empleado {
    private double bonificacion;

    public Gerente(String nombre, String documento,
                    double salarioBase, double bonificacion) {
        super(nombre, documento, salarioBase);
        this.bonificacion = bonificacion;
    }

    @Override
    public double calcularSalario() {
        return salarioBase + bonificacion;
    }
}
```

Archivo: MainEmpresa.java

```
public class MainEmpresa {
    public static void main(String[] args) {
        Vendedor v = new Vendedor("Ana Lopez", "1234", 1500000, 350000);
        Gerente g = new Gerente("Pedro Ruiz", "5678", 3000000, 800000);

        v.mostrarInfo();
        System.out.println("---");
        g.mostrarInfo();
    }
}
```

BENEFICIO APLICADO

El método *mostrarInfo()* está definido **solo en el padre**, pero funciona perfectamente con cualquier hijo. Cuando llama a *calcularSalario()*, Java automáticamente ejecuta la versión de la hija. Esto es **reutilización real**: una sola implementación sirve para Vendedor, Gerente y todos los empleados futuros.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Crea una tercera clase hija **AprendizSena** que extienda de **Empleado**. Un aprendiz gana el **50% del salario base** como apoyo de sostenimiento (según la normativa vigente). Sobrescribe *calcularSalario()* para reflejar esto. Agrega un aprendiz al main y prueba que *mostrarInfo()* funciona correctamente para los tres tipos.

Ejercicio 3: Concesionaria de Vehículos

SITUACIÓN PROBLEMA

Una concesionaria en Norte de Santander vende distintos tipos de vehículos. Todos tienen **marca**, **modelo** y **precio**. Pero además:

- Un **Carro** tiene un **número de puertas**.
- Una **Moto** tiene un **cilindraje**.

Cada tipo debe poder **describirse** mostrando su información específica.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Piensa:

1. ¿Puede una hija **reutilizar** el método del padre y también agregar más información?
2. ¿Qué palabra reservada usarías para llamar al método del padre desde la hija?
3. ¿Por qué es mejor esto que copiar y pegar el código del padre en la hija?

Solución guiada

Fíjate en las clases hijas: usan **super.describir()** para reutilizar el código del padre, y luego agregan su propia información específica. Es una técnica muy común y poderosa.

Archivo: Vehiculo.java

```
public class Vehiculo {
    protected String marca;
    protected String modelo;
    protected double precio;

    public Vehiculo(String marca, String modelo, double precio) {
        this.marca = marca;
        this.modelo = modelo;
        this.precio = precio;
    }

    public void describir() {
        System.out.println(marca + " " + modelo + " - $" + precio);
    }
}
```

Archivo: Carro.java

```
public class Carro extends Vehiculo {
    private int numeroPuertas;

    public Carro(String marca, String modelo, double precio, int puertas) {
        super(marca, modelo, precio);
        this.numeroPuertas = puertas;
    }

    @Override
    public void describir() {
        super.describir(); // Reutiliza el metodo del padre
        System.out.println("Tipo: Carro - " + numeroPuertas + " puertas");
    }
}
```

Archivo: Moto.java

```
public class Moto extends Vehiculo {
    private int cilindraje;

    public Moto(String marca, String modelo, double precio, int cilindraje) {
        super(marca, modelo, precio);
        this.cilindraje = cilindraje;
    }

    @Override
    public void describir() {
        super.describir();
        System.out.println("Tipo: Moto - " + cilindraje + " cc");
    }
}
```

Archivo: MainConcesionaria.java

```
public class MainConcesionaria {
    public static void main(String[] args) {
        Carro c = new Carro("Chevrolet", "Spark", 45000000, 4);
        Moto m = new Moto("Yamaha", "FZ", 12000000, 250);

        c.describir();
        System.out.println("---");
        m.describir();
    }
}
```

BENEFICIO APLICADO

El uso de *super.describir()* dentro de la hija muestra cómo se puede **extender** el comportamiento del padre sin reescribirlo. La hija agrega información, no reemplaza todo. Si mañana cambia la forma de mostrar la información básica, se cambia en un solo lugar (el padre) y todas las hijas se benefician automáticamente.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade una tercera hija **Camioneta** con un atributo **capacidadCarga** (en kg). Su método *describir()* debe llamar a *super.describir()* y agregar "Tipo: Camioneta - X kg de carga". Prueba las tres clases en el main.

Ejercicio 4: Figuras Geométricas

SITUACIÓN PROBLEMA

Un programa educativo para primaria necesita calcular áreas de figuras. Todas las figuras tienen algo en común: un **color**. Pero cada figura calcula su área de manera diferente:

- Un **Círculo** tiene **radio** y su área es $\pi \times r^2$.
- Un **Rectángulo** tiene **ancho y alto**, y su área es $\text{ancho} \times \text{alto}$.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Piensa:

1. ¿Tiene sentido que la clase padre *Figura* tenga un método *calcularArea()*? ¿Qué debería devolver por defecto?
2. ¿Por qué cada hija debe sobrescribir *calcularArea()*?
3. ¿Qué constante de Java se usa para representar el número π (pi)?

Solución guiada

Archivo: `Figura.java`

```
public class Figura {
    protected String color;

    public Figura(String color) {
        this.color = color;
    }

    public double calcularArea() {
        return 0; // Sera sobrescrito por las hijas
    }

    public String getColor() {
        return color;
    }
}
```

Archivo: Circulo.java

```
public class Circulo extends Figura {
    private double radio;

    public Circulo(String color, double radio) {
        super(color);
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```

Archivo: Rectangulo.java

```
public class Rectangulo extends Figura {
    private double ancho;
    private double alto;

    public Rectangulo(String color, double ancho, double alto) {
        super(color);
        this.ancho = ancho;
        this.alto = alto;
    }

    @Override
    public double calcularArea() {
        return ancho * alto;
    }
}
```

Archivo: MainFiguras.java

```
public class MainFiguras {
    public static void main(String[] args) {
        Circulo c = new Circulo("Rojo", 5);
        Rectangulo r = new Rectangulo("Azul", 4, 6);

        System.out.println("Circulo " + c.getColor() +
            " - Area: " + c.calcularArea());
        System.out.println("Rectangulo " + r.getColor() +
            " - Area: " + r.calcularArea());
    }
}
```

BENEFICIO APLICADO

Toda figura tiene un *color* (atributo común), pero cada una calcula su área de forma diferente. La **jerarquía clara** permite agregar fácilmente más figuras (Triángulo, Cuadrado, Trapecio) **sin tocar** lo existente. Este es el principio de **Open/Closed**: abierto a extensión, cerrado a modificación.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Crea una tercera clase hija **Triangulo** que reciba **base** y **altura**. El área de un triángulo es $(base \times altura) / 2$. Añade el triángulo al main. **Reto extra:** agrega un método *describir()* en la clase padre que muestre el color y el área, y aprovéchalo desde el main.

Ejercicio 5: Usuarios de una Plataforma Educativa

SITUACIÓN PROBLEMA

Una plataforma de cursos online (tipo SENA Sofía Plus) tiene **usuarios** con **nombre** y **correo**. Existen dos tipos principales:

- **Estudiante:** tiene una cantidad de **cursos inscritos**.
- **Instructor:** tiene una **especialidad** (área que enseña).

Ambos pueden **iniciarSesion()**, pero al hacerlo cada rol ve una pantalla distinta.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Reflexiona:

1. ¿El saludo "Bienvenido, nombre" debería estar en el padre o en cada hija?
2. ¿Cómo puedes hacer que la hija **llame al saludo del padre** y luego agregue información propia?
3. Si mañana se agrega el rol **Administrador**, ¿cuánto código tendrías que escribir?

Solución guiada

Archivo: **Usuario.java**

```
public class Usuario {
    protected String nombre;
    protected String correo;

    public Usuario(String nombre, String correo) {
        this.nombre = nombre;
        this.correo = correo;
    }

    public void iniciarSesion() {
        System.out.println("Bienvenido, " + nombre);
    }
}
```

Archivo: Estudiante.java

```
public class Estudiante extends Usuario {
    private int cursosInscritos;

    public Estudiante(String nombre, String correo, int cursos) {
        super(nombre, correo);
        this.cursosInscritos = cursos;
    }

    @Override
    public void iniciarSesion() {
        super.iniciarSesion(); // Saludo del padre
        System.out.println("Tienes " + cursosInscritos +
            " cursos disponibles para estudiar.");
    }
}
```

Archivo: Instructor.java

```
public class Instructor extends Usuario {
    private String especialidad;

    public Instructor(String nombre, String correo, String especialidad) {
        super(nombre, correo);
        this.especialidad = especialidad;
    }

    @Override
    public void iniciarSesion() {
        super.iniciarSesion();
        System.out.println("Panel del instructor de " + especialidad +
            " listo. Revisa tus estudiantes.");
    }
}
```

Archivo: MainPlataforma.java

```
public class MainPlataforma {
    public static void main(String[] args) {
        Estudiante e = new Estudiante("Carlos Vega",
            "carlos@sena.edu.co", 5);

        Instructor i = new Instructor("Marta Soto",
            "marta@sena.edu.co", "Java");

        e.iniciarSesion();
        System.out.println("---");
        i.iniciarSesion();
    }
}
```

BENEFICIO APLICADO

El método *iniciarSesion()* aprovecha el saludo del padre (común a todos) y agrega comportamiento específico de cada rol. Es un ejemplo claro de cómo la herencia **refleja la realidad**: todos los usuarios entran a la plataforma, pero cada rol ve cosas diferentes según sus responsabilidades.

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Añade una tercera clase hija **Administrador** con un atributo **nivelPermiso** (String: "TOTAL" o "PARCIAL"). Su *iniciarSesion()* debe llamar al padre y luego mostrar: "Panel de administración con permisos: [nivelPermiso]". Prueba los tres roles en el main.

Reto Integrador: Personal de una IPS

SITUACIÓN PROBLEMA

Una IPS necesita un sistema para su **personal**. Todo el personal tiene **documento**, **nombre** y **salario base**, y puede **registrarAsistencia()**. Existen tres tipos:

- **Medico:** tiene **especialidad**. Gana salario base + \$500.000 por cada consulta atendida (atributo **consultasDelMes**).
- **Enfermero:** tiene **turno** (mañana, tarde, noche). Gana salario base + \$200.000 si el turno es "noche".
- **Administrativo:** tiene **area** (recepción, facturación, etc.). Gana solo el salario base.

Todos deben poder **calcularSalarioMensual()** y **mostrarInfo()**.

ANTES DE PROGRAMAR — ACTIVA TU MENTE

Antes de resolver:

1. ¿Qué atributos y métodos van en la clase padre *PersonalIPS*?
2. ¿Qué método debería estar en el padre y ser sobrescrito por cada hija?
3. ¿El método *registrarAsistencia()* tiene que sobrescribirse o puede quedarse igual en el padre?

CONSTRUYE TÚ MISMO — RETO DE ANDAMIAJE

Este reto lo resuelves tú, aplicando **TODO** lo aprendido:

1. Diseña la clase padre **PersonalIPS.java** con los atributos `protected` y los métodos *calcularSalarioMensual()* y *mostrarInfo()*.
2. Diseña las tres hijas: **Medico.java**, **Enfermero.java**, **Administrativo.java**.
3. Cada hija debe usar *super()* en su constructor y sobrescribir *calcularSalarioMensual()*.
4. Diseña un **MainIPS.java** que:
 - Cree un médico con 20 consultas del mes
 - Cree un enfermero del turno noche
 - Cree un administrativo de facturación
 - Muestre la información de los tres (con su salario correcto)

Entrega: los cuatro archivos .java funcionando. Compara con un compañero cómo resolvieron ambos y discutan las diferencias.

Cierre metacognitivo del taller

REFLEXIÓN METACOGNITIVA — ¿QUÉ APRENDÍ?

Antes de terminar, tómate 5 minutos para responder en tu cuaderno:

1. Con tus propias palabras, ¿qué es la herencia? Usa una analogía distinta a la del cajero o los animales.
2. ¿En qué ejercicio la herencia te ahorró **más código repetido**?
3. Menciona **dos ejemplos de tu vida cotidiana** donde exista una relación "es un" (por ejemplo, "un smartphone es un teléfono").
4. ¿Por qué usar *protected* en vez de *private* en los atributos del padre?
5. ¿Qué diferencia hay entre *super()* y *this()*? (Investiga si no lo sabes).
6. ¿Qué duda te quedó? Anótala para consultarla con tu instructor.

Conclusión del taller

La herencia te permitió construir **jerarquías de clases organizadas**, reutilizar código, especializar comportamientos y modelar relaciones reales del tipo "**es un**". Esta práctica es el cimiento para entender el **polimorfismo**, que verás en el Taller 4. Recuerda: **usa herencia solo cuando exista una relación natural "es un"**, nunca para reutilizar código a la fuerza. En el próximo taller conocerás la **Abstracción**, que te enseñará a diseñar contratos claros entre clases.

SENA - Servicio Nacional de Aprendizaje
Formación Profesional Integral
Programa: Análisis y Desarrollo de Software